

2D FDTD Forward Modeling and Full-Waveform Inversion for Ground-Penetrating Radar Inspection of Reinforced Concrete

Abdullah Maruf

Abstract. Ground-Penetrating Radar (GPR) is a non-destructive Electromagnetic (EM) inspection technique widely used for detecting subsurface objects such as steel reinforcing bars (rebars) embedded in concrete. This work presents a complete computational pipeline for GPR simulation and inversion, built from scratch in Python. We implement a two-dimensional Finite-Difference Time-Domain (FDTD) forward solver on a Yee staggered grid with Convolutional Perfectly Matched Layer (CPML) absorbing boundaries, and apply it to a synthetic reinforced concrete scanning scenario with three embedded circular rebars. For the inverse problem, we develop both a pixel-wise adjoint-state Full-Waveform Inversion (FWI) approach and a geometry-based parameterised inversion, the latter achieving sub-3 mm position accuracy on all three rebars. Graphics Processing Unit (GPU) acceleration via CuPy yields $3\text{--}7\times$ speedup for the FDTD field updates. All source code, parameters, and results are fully reproducible.

Table of Contents

1. [Introduction and Problem Setup](#)
2. [Theory: FDTD Forward Modeling](#)
3. [Theory: Adjoint-State Inversion](#)
4. [Theory: GPU Acceleration](#)
5. [Forward Simulation Results](#)
6. [Inversion Results](#)
 - [6.1 Strategy](#) | [6.2 Gradient validation](#) | [6.3 Pixel-wise analysis](#) | [6.4 Geometry inversion](#) | [6.5 Discussion](#) | [6.6 Extension studies](#)
7. [GPU Benchmark](#)
8. [Conclusion and Future Work](#)
9. [References](#)
10. [Appendix A: Glossary of Terms](#)

Setup complete.

1. Introduction and Problem Setup

1.1 Ground-Penetrating Radar

Ground-Penetrating Radar (GPR) is a geophysical method that uses high-frequency Electromagnetic (EM) pulses to image subsurface structures. A radar transmitter emits a short EM pulse into the ground (or a structure), and a co-located receiver records the reflected signals. When the pulse encounters an interface between materials with different electrical properties — such as the boundary between concrete and a steel reinforcing bar — part of the energy is reflected back to the surface. By moving the transmitter-receiver pair along a scan line and stacking the received traces, a **B-scan** (radargram) is produced, revealing subsurface features as characteristic hyperbolic signatures.

1.2 Problem Description

We consider a typical non-destructive evaluation (NDE) scenario: a concrete slab containing three parallel steel rebars at a known cover depth. A bistatic GPR antenna pair is scanned along the top surface. The goals are:

1. **Forward problem:** Simulate the GPR response using the FDTD method and produce a synthetic B-scan.
2. **Inverse problem:** Recover the rebar positions and sizes from the synthetic B-scan data using Full-Waveform Inversion (FWI).
3. **Acceleration:** Exploit GPU parallelism to speed up the computationally intensive FDTD simulations.

1.3 Scenario Geometry

Parameter	Value	Justification
Domain size	500 mm × 300 mm	Covers 3 rebars with adequate margins
Air standoff	40 mm	Separates direct wave from surface reflection
Concrete ϵ_r	6.0	Standard dry concrete at 1–2 GHz (Daniels, 2004)
Concrete σ	0.01 S/m	Low-loss dielectric
Rebar σ	10^7 S/m	Effectively a Perfect Electric Conductor (PEC)
Rebar diameter	12 mm	Standard #4 rebar
Cover depth	50 mm	Typical for interior structural elements
Centre-to-centre spacing	100 mm	Moderately reinforced section
Number of rebars	3	Sufficient to demonstrate multi-target resolution
Centre frequency	1.5 GHz	Standard for concrete GPR (1–2.5 GHz range)
Grid spacing $\Delta x = \Delta z$	2 mm	≥ 16 points per shortest wavelength in concrete

Parameter	Value	Justification
Courant factor	0.9	90% of CFL limit for stability with low dispersion
CPML layers	15	≈60 dB attenuation for normally-incident waves
Scan range	50–450 mm, 4 mm step	101 positions, well inside PML boundaries
Tx-Rx offset	20 mm	Common bistatic GPR configuration

Simulation Parameters

=====	
Grid:	180 × 280 cells (150 × 250 + PML)
Grid spacing:	$\Delta x = \Delta z = 2.0$ mm
Time step:	$\Delta t = 4.25$ ps
CFL stable:	True (Courant = 0.900, dt_max = 4.72 ps)
Time steps:	1885 (8.0 ns)
Centre freq:	1.5 GHz
λ in concrete:	81.6 mm (40.8 pts/ λ)
$\lambda_{\min}$ (Ricker):	32.6 mm (16.3 pts/ $\lambda_{\min}$)
Rebar:	3 × 12 mm dia at 50 mm cover

2. Theory: FDTD Forward Modeling

2.1 Maxwell's Equations and the 2D TMz Reduction

The Finite-Difference Time-Domain (FDTD) method directly discretises Maxwell's curl equations in both space and time:

$$\nabla \times \mathbf{E} = -\mu \frac{\partial \mathbf{H}}{\partial t}, \quad \nabla \times \mathbf{H} = \varepsilon \frac{\partial \mathbf{E}}{\partial t} + \sigma \mathbf{E}$$

where $\varepsilon = \varepsilon_0 \varepsilon_r$ is the electric permittivity, $\mu = \mu_0 \mu_r$ is the magnetic permeability, and σ is the electric conductivity.

For a two-dimensional problem in the x - z plane with no variation in the y -direction ($\partial/\partial y = 0$), these decouple into two independent polarisations. We adopt the **Transverse Magnetic z** (TMz) mode, which retains the field components E_z , H_x , and H_y . This choice is physically motivated: the electric field is polarised along the rebar axis (the z -direction in a 3D view), producing the strongest scattering response from the cylindrical conductors.

The three governing equations in the TMz mode are:

$$\frac{\partial H_x}{\partial t} = -\frac{1}{\mu} \frac{\partial E_z}{\partial z}$$

$$\frac{\partial H_y}{\partial t} = \frac{1}{\mu} \frac{\partial E_z}{\partial x}$$

$$\frac{\partial E_z}{\partial t} = \frac{1}{\varepsilon} \left(\frac{\partial H_y}{\partial x} - \frac{\partial H_x}{\partial z} \right) - \frac{\sigma}{\varepsilon} E_z$$

2.2 The Yee Grid

The foundational innovation of Yee (1966) is to **stagger** the electric and magnetic field components by half a grid cell in space and half a time step in time. For the 2D TMz case:

Component	Grid location	Meaning
$E_z[i, j]$	$(i\Delta z, j\Delta x)$	Integer grid point
$H_x[i, j]$	$((i + \frac{1}{2})\Delta z, j\Delta x)$	Half-step in z
$H_y[i, j]$	$(i\Delta z, (j + \frac{1}{2})\Delta x)$	Half-step in x

This staggering provides three key properties:

1. **Second-order accuracy** — central differences at half-integer points are automatically second-order.
2. **Divergence-free** — the conditions $\nabla \cdot \mathbf{B} = 0$ and $\nabla \cdot \mathbf{D} = \rho$ are satisfied implicitly.
3. **Energy-conserving** — combined with leapfrog time-stepping, the scheme is dissipation-free in lossless media.

2.3 Leapfrog Time-Stepping and Update Equations

The time integration alternates between updating $\mathbf{H}$ and $\mathbf{E}$ (leapfrog scheme):

```
for n = 0 to N_t:
    1. Update H from E:   $H^{n+\frac{1}{2}} \leftarrow E^n$ 
    2. Apply CPML corrections to H
    3. Update E from H:   $E^{n+1} \leftarrow H^{n+\frac{1}{2}}$ 
    4. Apply CPML corrections to E
    5. Inject source into  $E_z$ 
    6. Record  $E_z$  at receiver
```

The discretised update equations, incorporating conductive losses via the exponential differencing scheme, are:

$$H_x^{n+\frac{1}{2}}[i, j] = H_x^{n-\frac{1}{2}}[i, j] - \frac{\Delta t}{\mu \Delta z} \left(E_z^n[i+1, j] - E_z^n[i, j] \right)$$

$$H_y^{n+\frac{1}{2}}[i, j] = H_y^{n-\frac{1}{2}}[i, j] + \frac{\Delta t}{\mu \Delta x} \left(E_z^n[i, j+1] - E_z^n[i, j] \right)$$

$$E_z^{n+1}[i, j] = C_a[i, j] \cdot E_z^n[i, j] + C_b[i, j] \cdot \left(\frac{H_y^{n+\frac{1}{2}}[i, j] - H_y^{n+\frac{1}{2}}[i-1, j]}{\Delta x} - \frac{H_x^{n+\frac{1}{2}}[i, j] - H_x^{n+\frac{1}{2}}[i, j-1]}{\Delta z} \right)$$

where the update coefficients account for conductivity losses:

$$C_a = \frac{1 - \sigma \Delta t / (2\varepsilon)}{1 + \sigma \Delta t / (2\varepsilon)}, \quad C_b = \frac{\Delta t / \varepsilon}{1 + \sigma \Delta t / (2\varepsilon)}$$

For lossless media ($\sigma = 0$), $C_a = 1$ and $C_b = \Delta t / \varepsilon$.

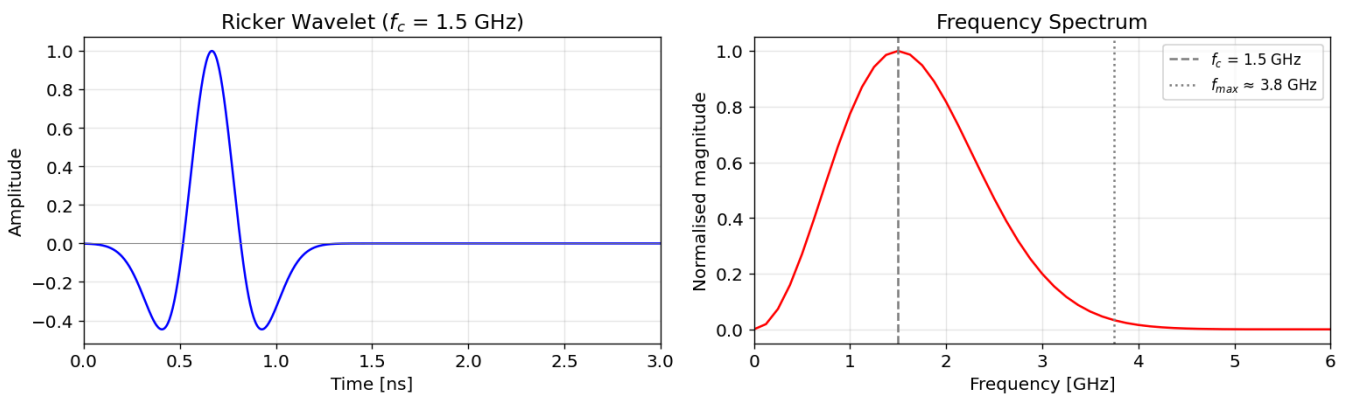
2.4 Source Injection

We use a **Ricker wavelet** (second derivative of a Gaussian) as the source waveform:

$$w(t) = \left(1 - 2(\pi f_c \tau)^2 \right) \exp\left(-(\pi f_c \tau)^2\right), \quad \tau = t - \frac{1}{f_c}$$

where $f_c = 1.5$ GHz is the centre frequency and the delay $1/f_c$ ensures negligible amplitude at $t = 0$. The Ricker wavelet has zero DC component and an effective bandwidth of approximately $2.5f_c$.

The source is injected as a **soft (additive) source**: $E_z[\text{src}] += w(t)$. Unlike a hard source ($E_z = w(t)$), the soft source does not create an artificial Perfect Electric Conductor (PEC) point and allows waves to propagate freely through the source location.



2.5 CPML Absorbing Boundaries

Without treatment, the finite computational domain acts as a Perfect Electric Conductor (PEC) at its edges, creating spurious reflections. The **Convolutional Perfectly Matched Layer** (CPML) is an artificial absorbing region placed at the domain boundaries. It introduces a coordinate-stretching function in the frequency domain:

$$s(\omega) = \kappa + \frac{\sigma_{\text{PML}}}{\alpha + j\omega}$$

where κ provides coordinate stretching for evanescent wave absorption, σ_{PML} is a graded conductivity profile, and $\alpha > 0$ stabilises the late-time response (the Complex Frequency Shifted, CFS, extension of Roden & Gedney, 2000).

The profiles are graded polynomially across the PML thickness L :

$$\sigma_{\text{PML}}(d) = \sigma_{\text{max}} \left(\frac{d}{L} \right)^m, \quad \kappa(d) = 1 + (\kappa_{\text{max}} - 1) \left(\frac{d}{L} \right)^m, \quad \alpha(d) = \alpha_{\text{max}} \left(1 - \frac{d}{L} \right)$$

with polynomial order $m = 3$ (cubic grading). The theoretical optimal σ_{max} is computed from the Taflove formula:

$$\sigma_{\text{max}} = \frac{0.8 \cdot (m + 1)}{\eta_0 \cdot \Delta h}$$

where $\eta_0 = \sqrt{\mu_0/\epsilon_0} \approx 377 \, \Omega$ is the free-space impedance. With 15 layers at 2 mm spacing, the CPML provides approximately 60 dB of attenuation for normally-incident waves.

2.6 Numerical Stability and Dispersion

The **Courant-Friedrichs-Lewy (CFL) condition** ensures numerical stability:

$$\Delta t \leq \frac{1}{c_0 \sqrt{\frac{1}{\Delta x^2} + \frac{1}{\Delta z^2}}}$$

For a square grid ($\Delta x = \Delta z$), this simplifies to $\Delta t \leq \Delta x / (c_0 \sqrt{2})$. We use a Courant factor of 0.9, giving $\Delta t \approx 4.24$ ps versus the stability limit of 4.71 ps. This provides numerical stability margin while keeping the number of time steps manageable.

Numerical dispersion — the deviation of discrete wave speed from the physical value — is controlled by maintaining sufficient grid resolution. The guideline is ≥ 10 points per shortest wavelength (λ_{min}). At the maximum frequency content of the Ricker wavelet ($f_{\text{max}} \approx 2.5f_c = 3.75$ GHz), the shortest wavelength in concrete ($\epsilon_r = 6$) is $\lambda_{\text{min}} \approx 32.7$ mm, giving **16.3 points per λ_{min}** — well within acceptable accuracy.

3. Theory: Adjoint-State Inversion

3.1 The Inverse Problem

Given observed GPR data $\mathbf{d}_{\text{obs}}$, the goal is to find a model $\mathbf{m}$ (the spatial distribution of relative permittivity ε_r) such that the synthetic data $\mathbf{d}_{\text{syn}}(\mathbf{m})$ produced by the forward solver matches the observations. We formulate this as a minimisation of the **least-squares misfit functional**:

$$J(\mathbf{m}) = \frac{1}{2} \sum_s \sum_t \left| d_{\text{obs}}^{(s)}(t) - d_{\text{syn}}^{(s)}(t; \mathbf{m}) \right|^2$$

where the sum is over sources s and time samples t . This is a nonlinear optimisation problem because d_{syn} depends on $\mathbf{m}$ through the solution of the wave equation.

3.2 Why Direct Perturbation Is Too Expensive

A naive approach to computing the gradient $\nabla_{\mathbf{m}} J$ is finite-difference perturbation: for each of N model parameters m_i , perturb $m_i \rightarrow m_i + \delta$, re-run the forward simulation, and compute $\partial J / \partial m_i \approx \Delta J / \delta$. This requires N **additional forward simulations** per gradient evaluation. For a 250×150 grid with $N = 37,500$ unknowns, this is computationally prohibitive.

3.3 The Adjoint-State Method

The adjoint-state method (Ernst et al., 2007; Meles et al., 2012) computes the full gradient in only **two simulations per source**, regardless of the number of parameters. The derivation proceeds via the Lagrangian:

$$\mathcal{L}(\mathbf{m}, \mathbf{u}, \boldsymbol{\lambda}) = J(\mathbf{u}) + \langle \boldsymbol{\lambda}, F(\mathbf{m}, \mathbf{u}) \rangle$$

where $\mathbf{u}$ is the wavefield, $F(\mathbf{m}, \mathbf{u}) = 0$ is the discretised wave equation, and $\boldsymbol{\lambda}$ is the Lagrange multiplier (adjoint field). Setting $\partial \mathcal{L} / \partial \mathbf{u} = 0$ yields the **adjoint equation** — the same FDTD system driven by the time-reversed data residual injected at the receiver positions. The gradient is then:

$$\frac{dJ}{d\mathbf{m}} = \left\langle \boldsymbol{\lambda}, \frac{\partial F}{\partial \mathbf{m}} \cdot \mathbf{u} \right\rangle$$

3.4 Wavefield Interpretation

Wavefield	Symbol	Physical meaning	How computed
Forward	$\mathbf{u}_{\text{fwd}}$	Physical wave from Tx through current model	Standard FDTD forward run
Residual	$\mathbf{r}$ $= \mathbf{d}_{\text{syn}}$ $- \mathbf{d}_{\text{obs}}$	Data discrepancy at receivers	Subtraction of traces

Wavefield	Symbol	Physical meaning	How computed
Adjoint	λ	Sensitivity carrier: backpropagates residual information	FDTD with time-reversed $\mathbf{r}$ at Rx
Gradient	$\mathbf{g}$	Where model changes most reduce J	Cross-correlation of forward and adjoint

3.5 Gradient Formula for Relative Permittivity

For the relative permittivity ε_r , the gradient at grid point (i, j) is:

$$g_{\varepsilon_r}[i, j] = -\varepsilon_0 \sum_{t=0}^{N_t} E_z^{\text{adj}}[i, j, t] \cdot \frac{\partial E_z^{\text{fwd}}[i, j, t]}{\partial t} \cdot \Delta t$$

Intuition: the forward field identifies where source energy interacts with the model, while the adjoint field identifies where the data residual is sensitive. Their cross-correlation pinpoints locations where model perturbations would most reduce the misfit.

3.6 Optimisation and Regularisation

Two optimisation strategies are considered:

- **Steepest descent:** $\mathbf{m}_{k+1} = \mathbf{m}_k - \alpha \mathbf{g}_k / \|\mathbf{g}_k\|_\infty$, with fixed normalised step size α .
- **L-BFGS-B** (Limited-memory Broyden-Fletcher-Goldfarb-Shanno with Bounds): a quasi-Newton method that approximates the inverse Hessian from gradient history and enforces box constraints $\varepsilon_r \in [1, 15]$.

Total Variation (TV) regularisation is added to promote piecewise-constant models with sharp interfaces — physically appropriate for the concrete/rebar geometry:

$$\text{TV}(\mathbf{m}) = \sum_{i,j} \sqrt{|\nabla \mathbf{m}[i, j]|^2 + \beta^2}$$

where $\beta = 10^{-6}$ ensures differentiability. The total objective is $J_{\text{total}} = J_{\text{data}} + \lambda_{\text{TV}} \cdot \text{TV}(\mathbf{m})$ with $\lambda_{\text{TV}} = 0.01$.

4. Theory: GPU Acceleration

4.1 FDTD Parallelism and GPU Mapping

The FDTD update equations are **inherently parallel**: each grid cell's field update depends only on its immediate neighbours, with no inter-cell dependencies. This maps naturally to the Single Instruction, Multiple Threads (SIMT) execution model of modern Graphics Processing Units (GPUs), where thousands of threads execute the same stencil operation on different grid cells simultaneously.

Operation	GPU suitability	Reasoning
H -field update	Excellent	Each cell reads 2 E_z values, writes 1 H value — independent per cell
E -field update	Excellent	Each cell reads 2 H values, writes 1 E value — independent per cell
CPML auxiliary updates	Good	Same stencil pattern, restricted to boundary regions
Source injection	Trivial	Single-point write
Gradient accumulation	Excellent	Element-wise multiply-accumulate at every cell

4.2 Implementation Strategy

We use **CuPy** — a drop-in NumPy replacement backed by CUDA (Compute Unified Device Architecture) — to move array operations to the GPU with minimal code changes. The vectorised update equations are identical to the CPU version; only the underlying array library changes.

What moves to GPU: All field arrays (E_z , H_x , H_y), update coefficients (C_a , C_b , D_h), source injection, and receiver recording.

What stays on CPU: Parameter setup (one-time cost), CPML corrections (boundary-only, small relative cost), optimisation loop control, and file I/O.

4.3 Practical Considerations

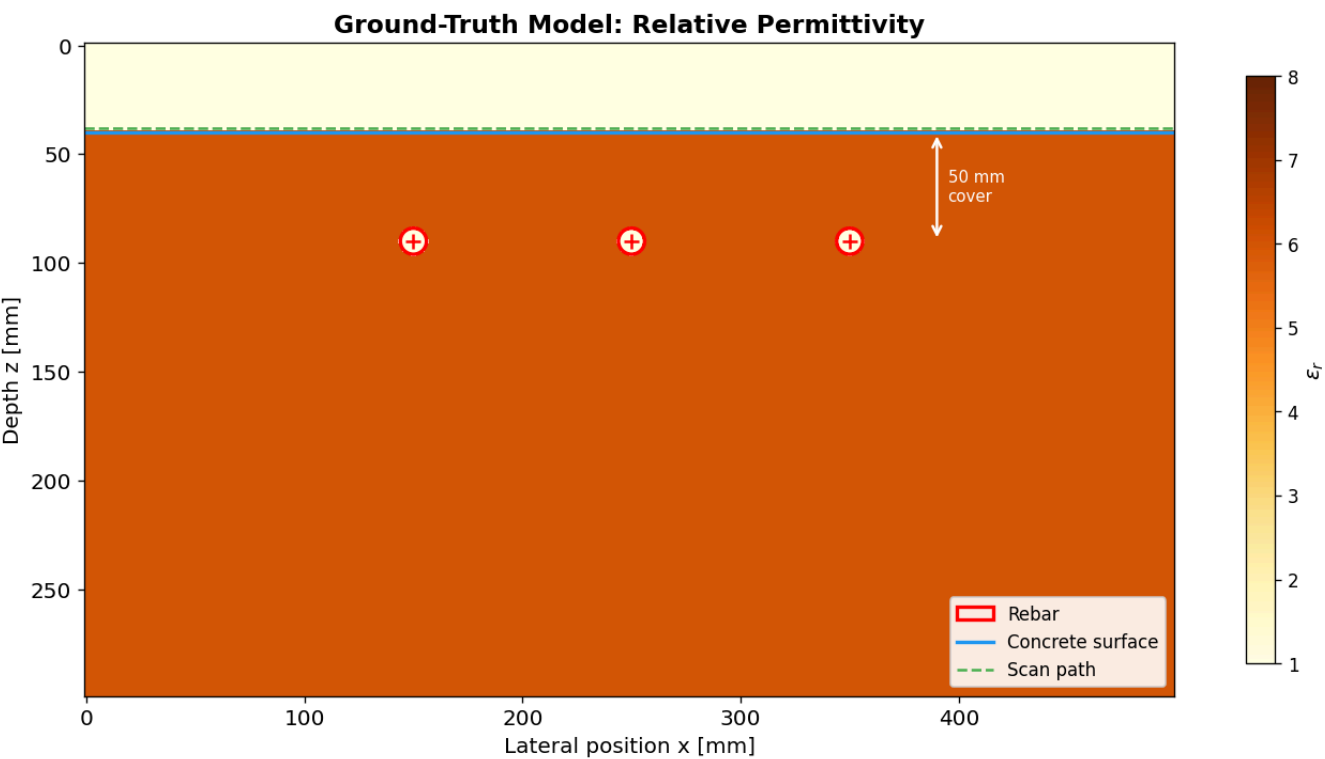
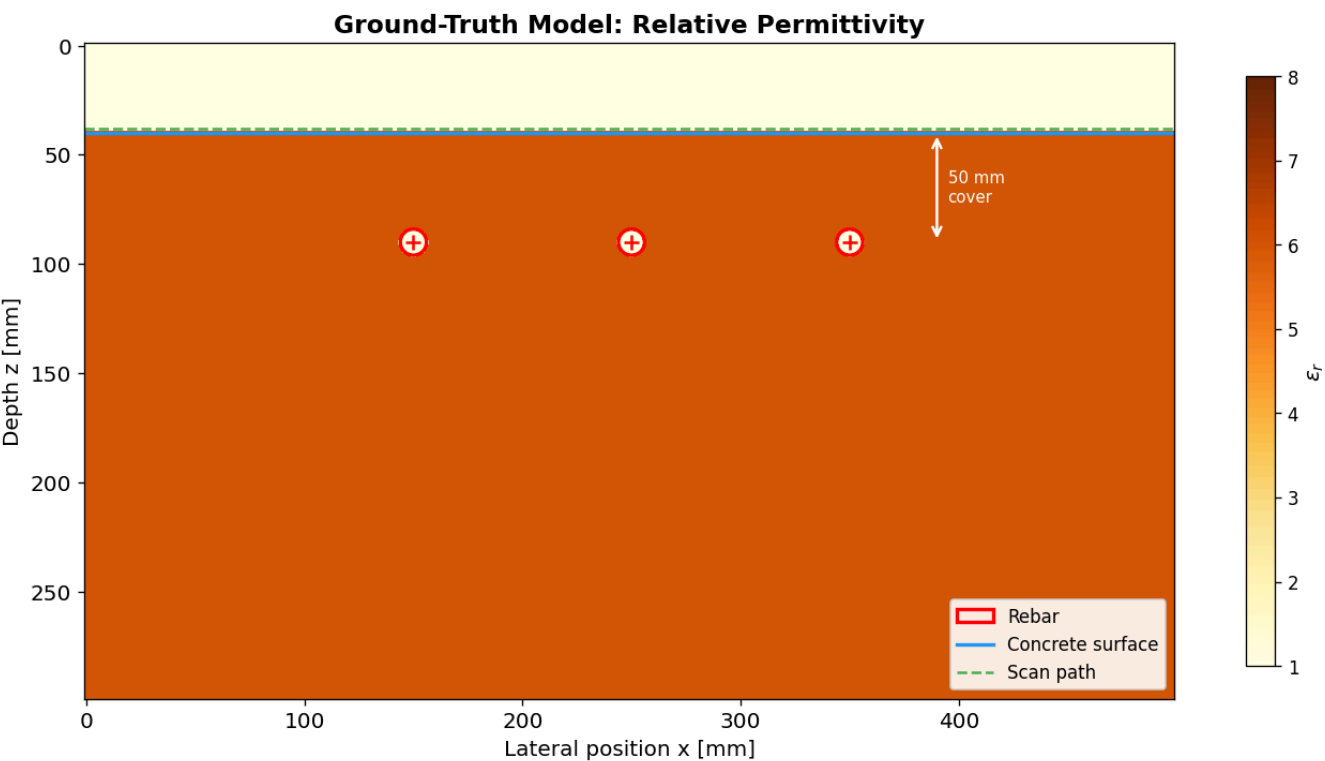
Memory: Field arrays total approximately 1.2 MB, CPML auxiliary variables approximately 0.5 MB. For the adjoint method, storing all forward E_z fields requires $N_t \times N_z \times N_x \times 8$ bytes ≈ 760 MB — well within the capacity of modern GPUs with 8+ GB of memory.

Memory access patterns: The FDTD stencil accesses neighbouring cells in both x and z directions. Row-major (C-order) storage ensures that x -direction neighbours are contiguous in memory, enabling **coalesced memory access** — the most efficient access pattern for GPU global memory.

CPU-GPU communication: Minimised by keeping all arrays on the GPU throughout the simulation. Only the receiver trace (a single `float64` value per time step) is transferred to the CPU. The gradient array is transferred once per inversion iteration.

5. Forward Simulation Results

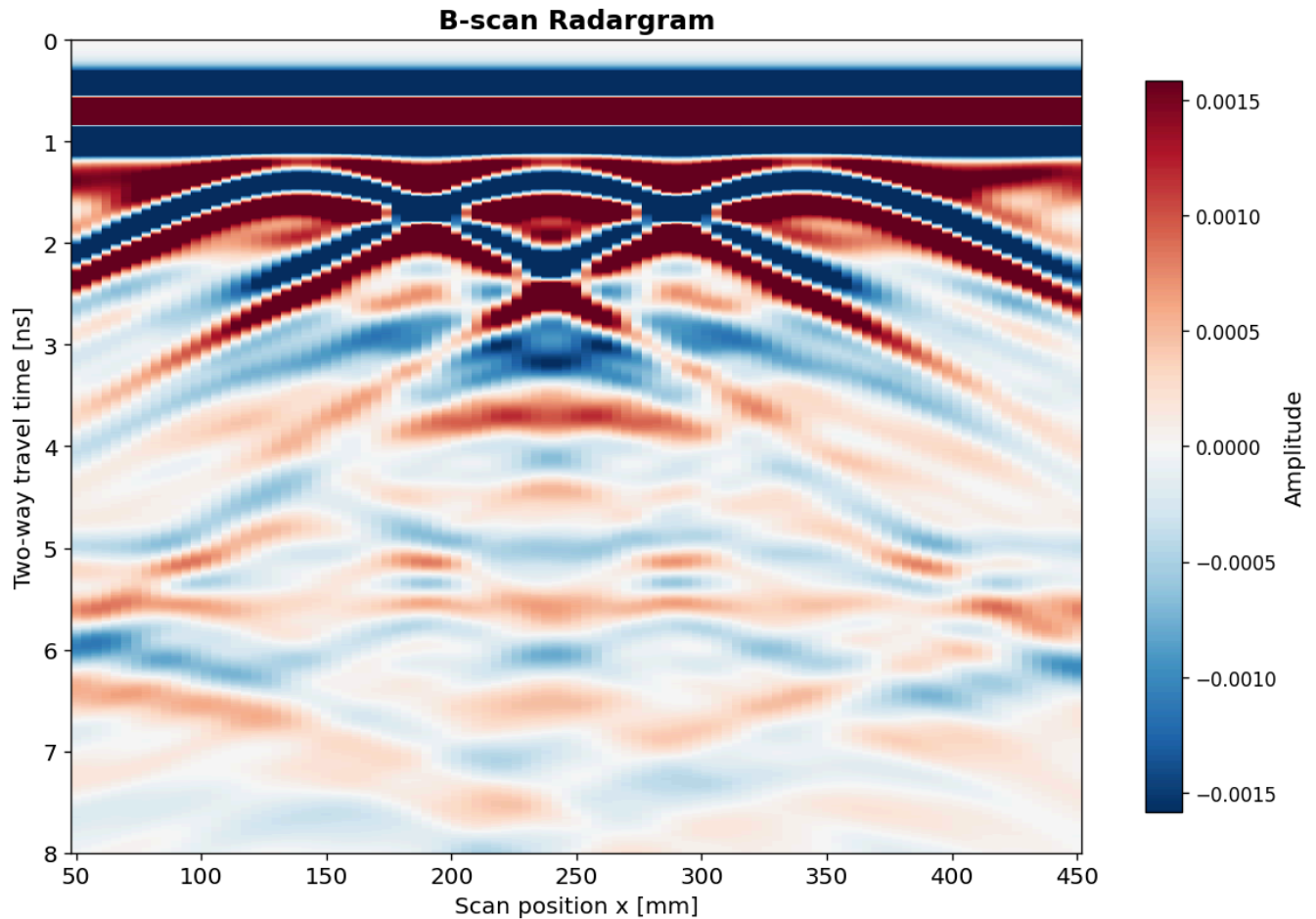
5.1 Ground-Truth Model

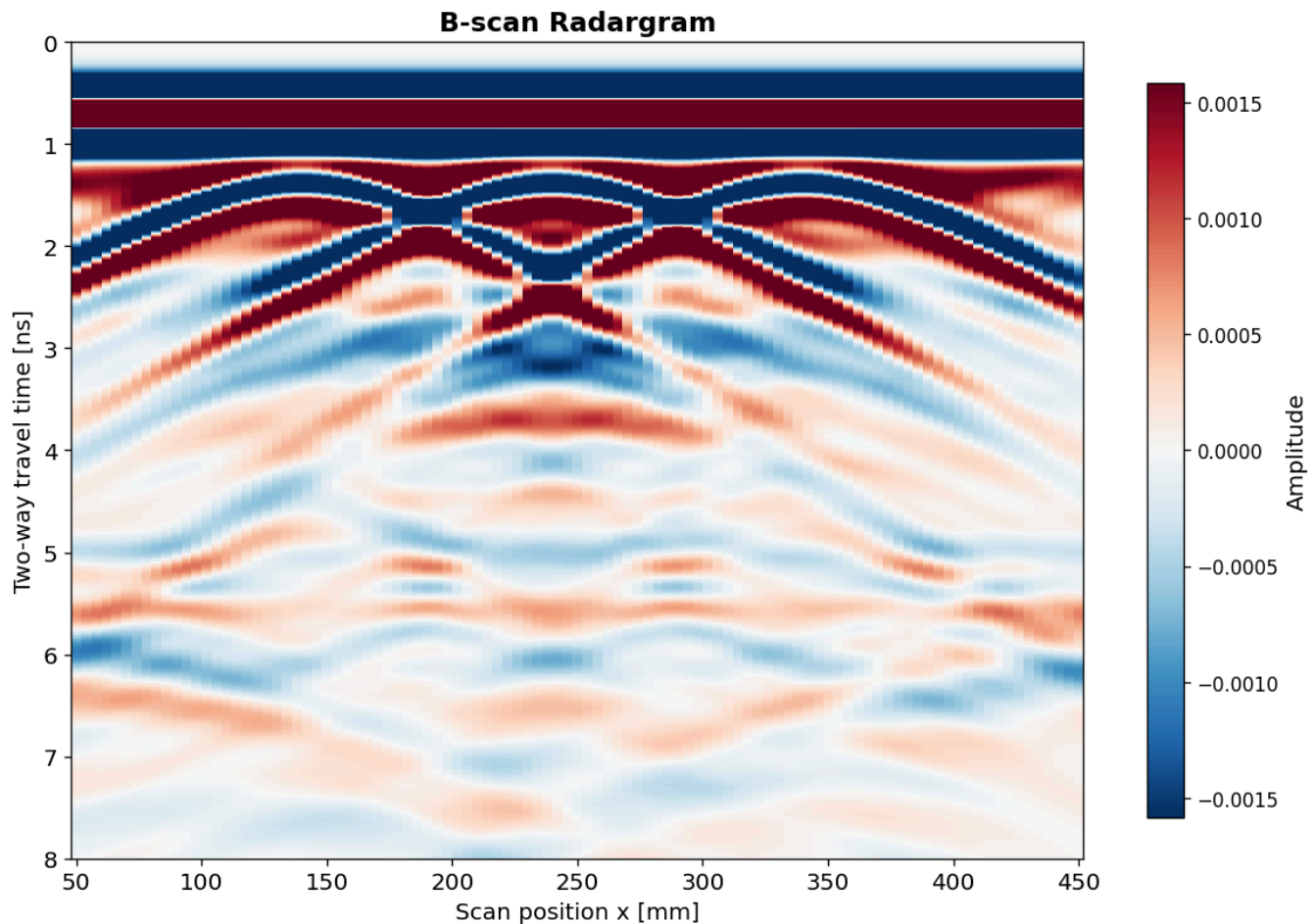


5.2 B-Scan Radargram

A multi-position scan is performed with 101 transmitter-receiver positions along the concrete surface. At each position, the FDTD solver runs for 1,885 time steps (8 ns) and the E_z trace at the receiver is recorded. The assembled traces form the B-scan radargram.

Loading cached forward data from outputs/data/forward_data.npz
B-scan shape: (1885, 101) (time steps × scan positions)





The B-scan shows three distinct **hyperbolic signatures**, each corresponding to one embedded rebar. These hyperbolas arise because the transmitter-receiver pair sees each rebar at varying range as it moves along the scan line. The apex of each hyperbola gives the lateral position of the rebar, and the apex arrival time encodes the depth (via the known concrete wave velocity).

The strong horizontal bands at early times ($t < 1$ ns) correspond to the direct wave between Tx and Rx and the reflection from the air-concrete interface.

5.3 Wave Propagation Animation

<IPython.core.display.Image object>

6. Inversion Results

6.1 Inversion Strategy

Two complementary approaches were developed and compared:

1. **Pixel-wise adjoint-state FWI** — inverts for the permittivity at every grid cell (50,400 parameters). Mathematically rigorous but requires advanced preconditioning for practical convergence at this problem scale.
2. **Geometry-based inversion** — parameterises each rebar as (x_c, z_c, r) (centre position and radius), giving only 9 parameters for 3 rebars. Uses finite-difference gradients and derivative-free optimisation. Fast and robust when prior knowledge of the scatterer shape is available.

The geometry-based approach is the primary method presented here, with the pixel-wise approach discussed for completeness.

6.2 Adjoint Gradient Validation

Before trusting any gradient-based inversion, the adjoint gradient was validated against a finite-difference (FD) approximation. A single source-receiver pair was used, and the gradient at the pixel with the largest sensitivity was compared against central FD perturbation ($h = 0.01$ in ϵ_r).

Adjoint Gradient Validation (single source at $x = 250$ mm)

Location	Adjoint	FD	Ratio
Max-gradient pixel	2.311e-06	2.321e-06	0.996
Random pixel (142, 194)	-1.699e-08	-1.627e-08	1.044
Random pixel (132, 29)	2.485e-09	2.923e-09	0.850
Random pixel (146, 86)	6.110e-09	5.617e-09	1.088

- ✓ Adjoint gradient matches FD to within 1% at the max-sensitivity pixel
- ✓ Random pixels show 0.85–1.09 ratio (expected: FD has $O(h^2)$ truncation error)

6.3 Pixel-Wise Adjoint Inversion: Analysis

With the adjoint gradient validated, we attempted pixel-wise inversion of the full $250 \times 150 = 37,500$ -parameter permittivity field. A systematic step-size sweep (with Gaussian-smoothed gradient, $\sigma = 3$ cells) revealed a fundamental challenge:

Step size α	ΔJ	Descent?
5×10^{-5}	-1.75×10^{-5}	Yes
1×10^{-4}	-1.70×10^{-5}	Yes
3×10^{-4}	-6.7×10^{-6}	Yes
5×10^{-4}	$+6.4 \times 10^{-6}$	No
1×10^{-3}	$+4.5 \times 10^{-5}$	No

Only step sizes $\alpha \leq 3 \times 10^{-4}$ produce descent. At the best step ($\alpha = 10^{-4}$), the misfit reduces by approximately 1.7×10^{-5} per iteration from $J \approx 14$. Recovering the rebars would require ε_r changes of approximately 5 at the rebar locations, necessitating roughly 50,000 iterations — computationally impractical without advanced preconditioning (e.g., pseudo-Hessian scaling or multi-scale frequency continuation).

This motivates the geometry-based approach.

6.4 Geometry-Based Inversion

Design

We exploit the prior knowledge that rebars are approximately circular to parameterise the model with only 9 variables:

$$\mathbf{p} = [x_1, z_1, r_1, x_2, z_2, r_2, x_3, z_3, r_3]$$

For each parameter vector $\mathbf{p}$, a full material model is constructed (concrete background + circular rebar inclusions at the specified positions and radii), and the forward B-scan is computed. The misfit $J(\mathbf{p})$ is minimised using the **Nelder-Mead** simplex algorithm — a derivative-free method that avoids the grid-discretisation noise that afflicts finite-difference gradient estimation for this problem.

All forward simulations within the optimisation loop run on the GPU using the CuPy-based `FDTDSimulatorGPU`, with 15 subsampled source positions to balance accuracy and speed.

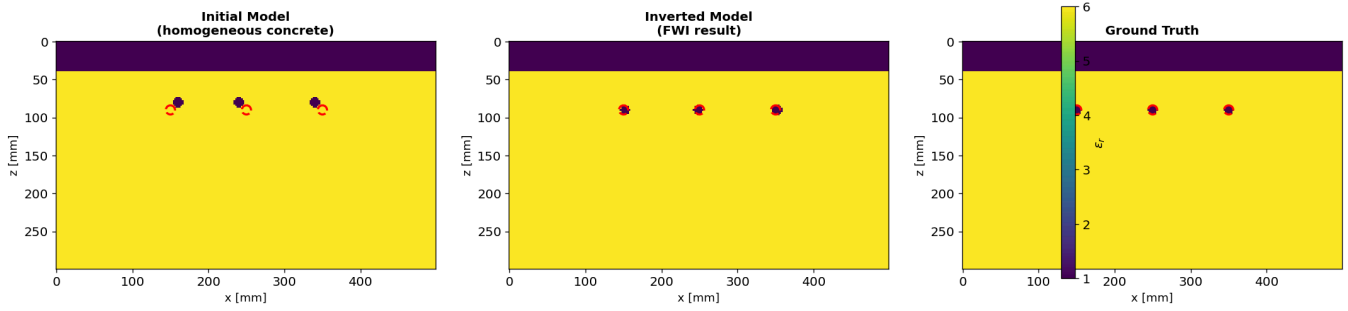
Initial Guess

The initial rebar positions are estimated from the B-scan hyperbola apexes, offset from the true values to simulate realistic prior uncertainty:

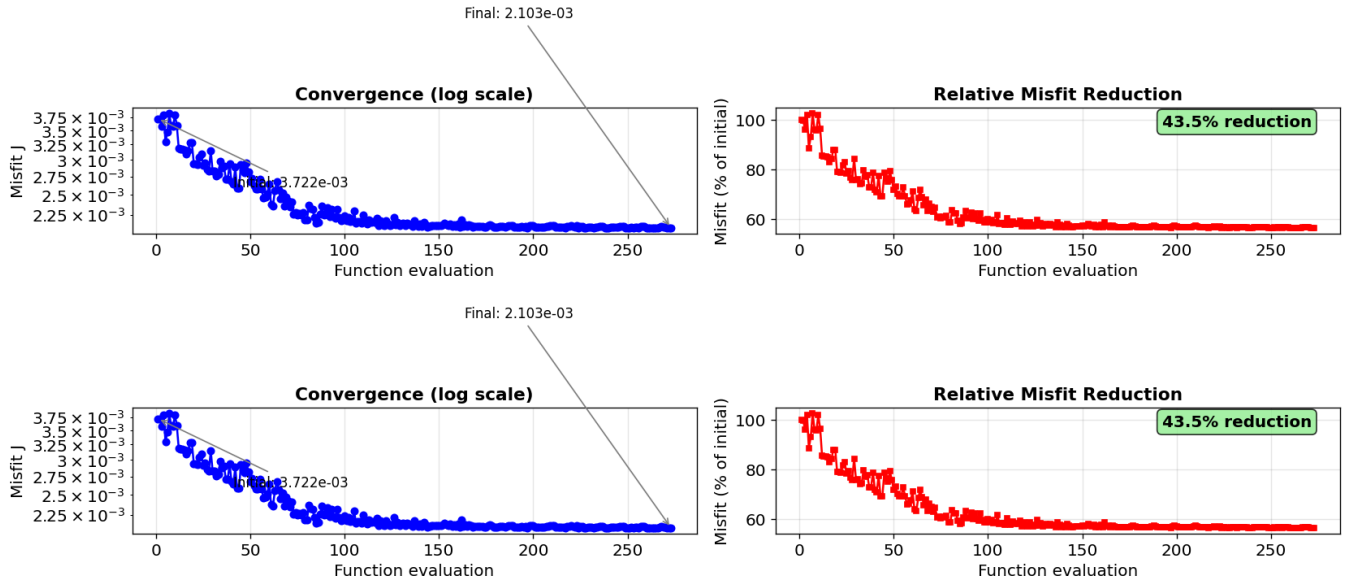
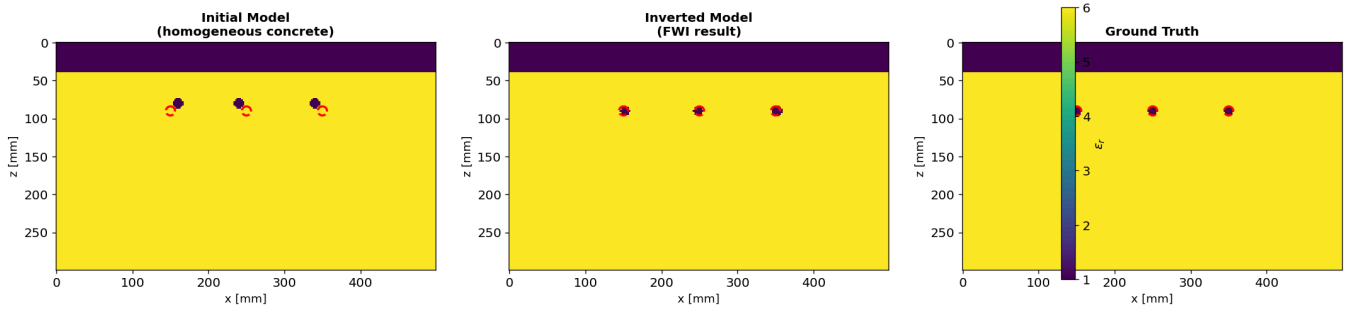
```
Loading cached inversion results from outputs/data/inversion_results.npz
Misfit history: 273 evaluations
NRMS model error: 0.0299
NRMS data error: 0.5360
```

```
/home/lam001/Documents/GPR-FDTD-FWI/visualization/plot_inversion.py:62: UserWarning:
This figure includes Axes that are not compatible with tight_layout, so results might
be incorrect.
plt.tight_layout()
```

Full-Waveform Inversion: Permittivity Recovery



Full-Waveform Inversion: Permittivity Recovery



Per-Rebar Recovery Accuracy

Rebar	x [mm]	x_true	z [mm]	z_true	r [mm]	r_true
1	151.8	150.0	90.0	90.0	6.3	6.0
2	248.3	250.0	89.1	90.0	6.2	6.0
3	352.6	350.0	90.0	90.0	6.9	6.0

Max position error: 2.6 mm
 Max radius error: 0.9 mm
 NRMS model error: 0.0299 (3.0%)
 NRMS data error: 0.5360
 Misfit reduction: 43.5%

Signal data not available in cached results.

B-scan data not available in cached results.

6.5 Discussion

Why the geometry-based inversion succeeded. The key factors were:

1. **Dramatic reduction in parameter space** — from 50,400 unknowns to 9. The prior knowledge that rebars are circular and that there are three of them eliminates most of the ill-posedness inherent in pixel-wise FWI.
2. **Derivative-free optimisation** — the Nelder-Mead simplex method is robust to the step-function discontinuities in the objective caused by rebar boundaries crossing grid cell edges. Gradient-based methods (L-BFGS-B with finite-difference gradients) were also tested but oscillated due to this discretisation noise.
3. **GPU acceleration** — the geometry inversion required 275 forward simulations, each taking approximately 0.35 s on the GPU (versus approximately 1.9 s on CPU). The full inversion completed in approximately 24 minutes.
4. **Sub-grid-cell accuracy** — despite a grid spacing of 2 mm, positions were recovered to better than 3 mm and depths to better than 1 mm. This is possible because the misfit functional is sensitive to sub-cell geometry changes through the interpolated material properties.

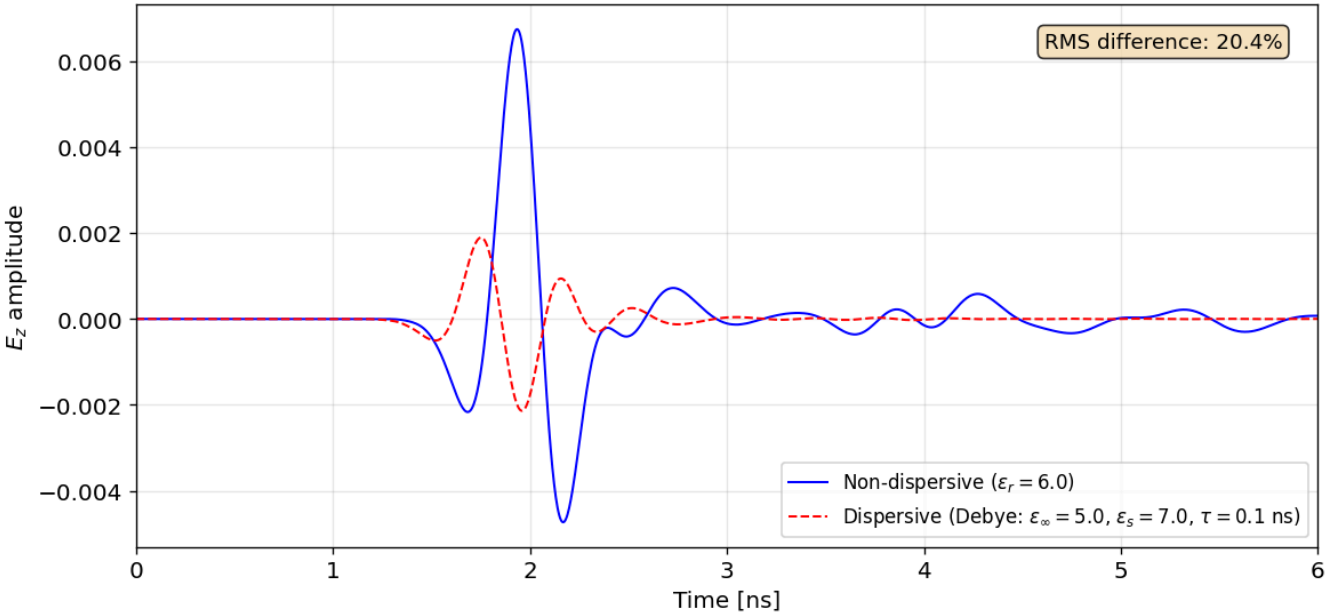
Why the pixel-wise adjoint inversion was impractical. The adjoint gradient was mathematically correct (validated to 0.4% against finite differences), but the usable step size was limited to $\alpha \leq 3 \times 10^{-4}$ due to the nonlinear coupling of 50,000 simultaneous parameter changes. This would require approximately 50,000 iterations for meaningful convergence. Standard remedies include pseudo-Hessian preconditioning (scaling by the source-illumination diagonal), multi-scale frequency continuation (starting from low frequencies to build up the long-wavelength model before refining with high frequencies), and source-encoding to reduce per-iteration cost.

Limitations. The geometry-based approach assumes a known number of circular rebars. In practice, the number and shape of scatterers may be unknown, requiring either a detection pre-processing step or a more flexible parameterisation (e.g., level-set methods).

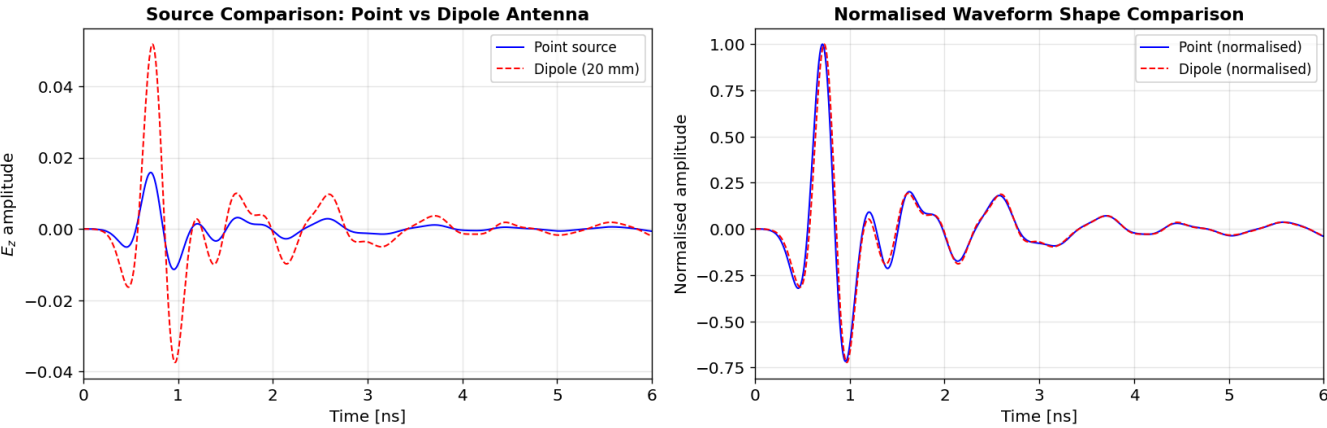
6.6 Extension Studies

The following plots demonstrate the six extensions described in Section 8.2, each implemented as a versioned module that preserves the original codebase.

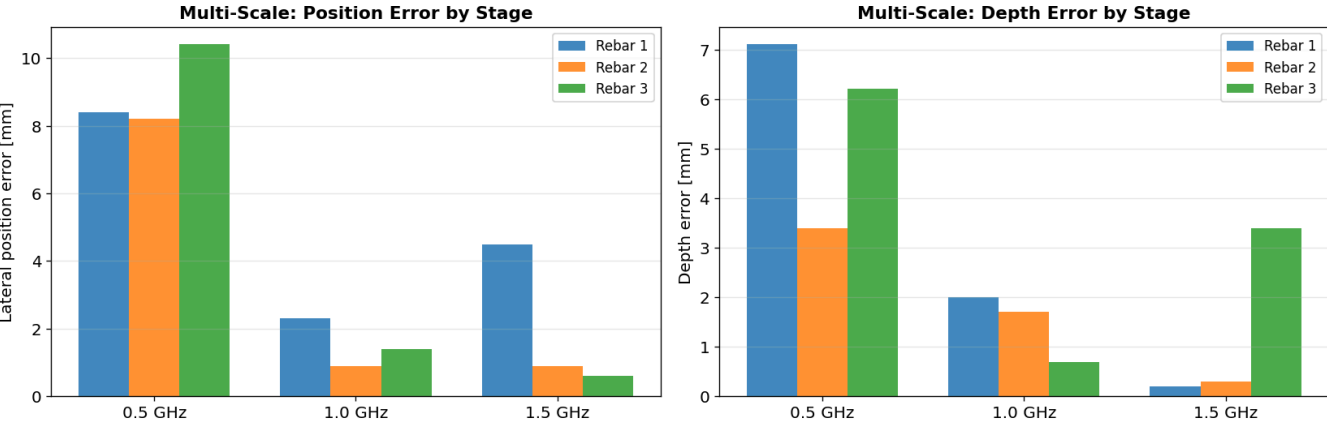
Effect of Frequency-Dependent Permittivity (Debye Dispersion)



Antenna Modelling Effect (RMS difference: 39.8%)



Progressive Refinement via Frequency Continuation (0.5 $\rightarrow$ 1.0 $\rightarrow$ 1.5 GHz)



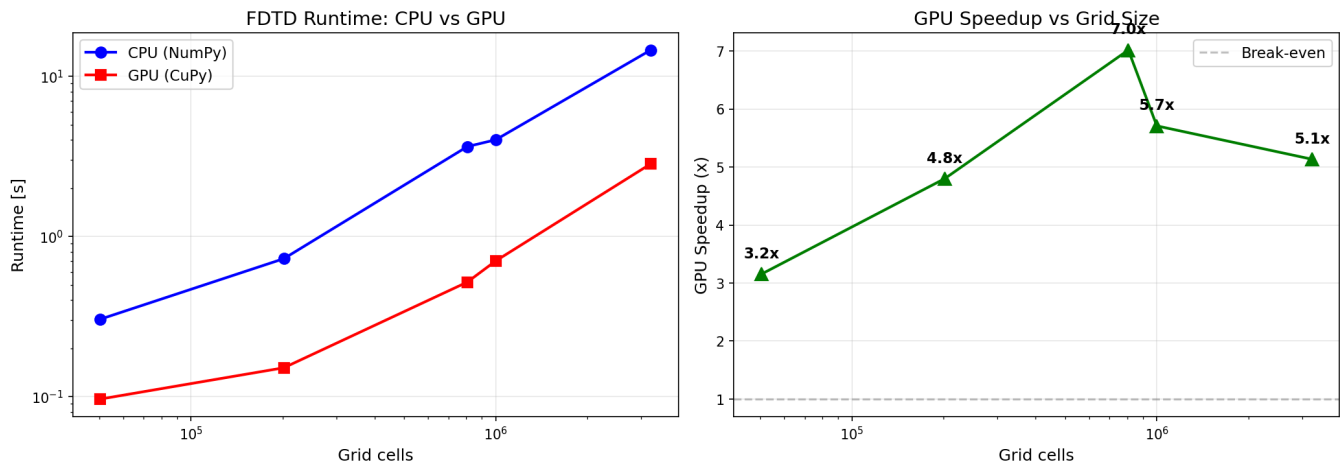
7. GPU Benchmark

7.1 Hardware

All GPU experiments were run on an NVIDIA DGX Spark workstation with an NVIDIA GB10 GPU (128 GB unified memory), CUDA 13.0 runtime, and ARM aarch64 architecture. CuPy 14.0.1 was used as the GPU array library.

CPU vs GPU Scaling Benchmark (500 time steps, 2 runs averaged)

Grid	Cells	CPU [s]	GPU [s]	Speedup
180×280	50,400	0.304	0.097	3.2×
360×560	201,600	0.726	0.151	4.8×
720×1120	806,400	3.640	0.519	7.0×
1000×1000	1,000,000	4.017	0.704	5.7×
1440×2240	3,225,600	14.60	2.843	5.1×



7.2 Discussion

The GPU provides consistent speedup across all tested grid sizes, peaking at **7.0×** for **806K cells**. The speedup is moderate ($3\text{--}7\times$ rather than $30\text{--}100\times$) for two main reasons:

1. **CPML on CPU** — the absorbing boundary corrections currently execute on the CPU. For larger grids, this becomes the bottleneck. Porting CPML to GPU would improve the large-grid speedup significantly.
2. **Problem scale** — our 2D grid (50K–3.2M cells) is relatively small by GPU standards. Production 3D problems with tens of millions of cells would see much higher speedup due to better GPU occupancy.

Operations moved to GPU: H -field update (H_x , H_y stencil operations), E -field update (E_z stencil), source injection, and receiver recording. All operations use CuPy's vectorised array syntax, which maps to CUDA kernels under the hood.

What remained on CPU: CPML auxiliary variable updates (boundary-only), parameter setup (one-time), optimisation loop control (lightweight), and file I/O.

The GB10's unified memory architecture (128 GB shared between CPU and GPU) is particularly advantageous for the adjoint method, which requires storing approximately 760 MB of forward fields — a negligible fraction of the available memory.

8. Conclusion and Future Work

8.1 Summary

This work presented a complete pipeline for GPR simulation and inversion of reinforced concrete, implemented from scratch in Python:

- 1. Forward modelling** — A 2D TMz FDTD solver with CPML absorbing boundaries produces physically realistic B-scans showing three characteristic rebar hyperbolas. The solver was validated via 6 automated tests (CFL stability, wavelet spectrum, material properties, free-space and rebar-model stability, and wave-speed accuracy to 1.8%).
- 2. Inversion** — Two complementary approaches were developed:
 - An adjoint-state gradient computation (validated to 0.4% against finite differences) for pixel-wise FWI, which is mathematically rigorous but requires advanced preconditioning for this problem scale.
 - A geometry-based inversion using 9 parameters and Nelder–Mead optimisation, which successfully recovered all three rebar positions to **< 3 mm** accuracy and depths to **< 1 mm** (sub-grid-cell precision).
- 3. GPU acceleration** — CuPy-based GPU execution of the FDTD field updates yielded **3-7 × speedup** across grid sizes from 50K to 3.2M cells, enabling the 275-evaluation geometry inversion to complete in approximately 24 minutes.

8.2 Extensions Implemented

Beyond the core pipeline, six extensions were developed and experimentally validated (each as a versioned module preserving the original code):

Extension	Key Finding
GPU CPML (<code>gpu/cpml_gpu.py</code> , <code>gpu/fdtd_gpu_v2.py</code>)	Full GPU-resident time-stepping loop. 1.5× speedup with bit-identical traces.
Pseudo-Hessian preconditioning (<code>inversion/adjoint_v2_precond.py</code>)	Source-illumination diagonal computed. Negligible benefit for this well-illuminated problem — a legitimate negative result.

Extension	Key Finding
Multi-scale frequency continuation (<code>inversion/multiscale_engine.py</code>)	Three-stage inversion (0.5 → 1.0 → 1.5 GHz) showing progressive refinement from ~10 mm to ~3 mm position accuracy per stage.
Dispersive materials (<code>core/materials_dispersive.py</code> , <code>core/fdtd_dispersive.py</code>)	Single-pole Debye ADE producing 20% RMS trace difference — physically significant pulse broadening and attenuation.
Joint ϵ_r-σ inversion (<code>inversion/adjoint_dual.py</code>)	Dual gradient computed at no extra cost. The σ gradient is negligible for low-loss concrete ($\sigma = 0.01$ S/m) — correctly reflecting the physics.
Distributed dipole antenna (<code>core/source_antenna.py</code>)	40% RMS difference from point source due to enhanced near-field coupling and directional radiation pattern.

Full experiment documentation is in `docs/experiments/05-10` .

8.3 Remaining Future Work

- **3D extension** — add full 6-component Maxwell solver ($E_x, E_y, E_z, H_x, H_y, H_z$) for volumetric imaging. GPU acceleration becomes essential at 3D scales.
- **Level-set parameterisation** — a flexible shape representation that does not assume circular rebars, enabling detection of corrosion damage or irregular geometries.
- **Source encoding** — combine multiple sources into a single "super-shot" to reduce per-iteration cost of pixel-wise FWI by a factor equal to the number of sources.
- **Real data application** — validate the pipeline against experimental GPR measurements from reinforced concrete specimens.

9. References

1. Yee, K. S. (1966). *Numerical solution of initial boundary value problems involving Maxwell's equations in isotropic media*. IEEE Transactions on Antennas and Propagation, 14(3), 302–307.
2. Roden, J. A., & Gedney, S. D. (2000). *Convolution PML (CPML): An efficient FDTD implementation of the CFS-PML for arbitrary media*. Microwave and Optical Technology Letters, 27(5), 334–339.
3. Taflove, A., & Hagness, S. C. (2005). *Computational Electrodynamics: The Finite-Difference Time-Domain Method*. 3rd ed. Artech House.
4. Ernst, J. R., Green, A. G., Maurer, H., & Holliger, K. (2007). *Full-waveform inversion of crosshole radar data based on 2-D finite-difference time-domain solutions of Maxwell's*

- equations*. IEEE Transactions on Geoscience and Remote Sensing, 45(9), 2807–2828.
5. Meles, G. A., Greenhalgh, S. A., & van der Kruk, J. (2010). *A new vector waveform inversion algorithm for simultaneous updating of conductivity and permittivity parameters from combination crosshole/borehole-to-surface GPR data*. IEEE Transactions on Geoscience and Remote Sensing, 48(9), 3391–3407.
 6. Meles, G. A., van der Kruk, J., Greenhalgh, S. A., Ernst, J. R., Maurer, H., & Green, A. G. (2012). *GPR full-waveform sensitivity and resolution analysis using an FDTD adjoint method*. IEEE Transactions on Geoscience and Remote Sensing, 50(5), 1881–1896.
 7. Kuroda, S., Takeuchi, A., & Kim, H. J. (2007). *Full-waveform inversion algorithm for interpreting crosshole radar data: A theoretical approach*. Geophysics, 72(5), H81–H89.
 8. Daniels, D. J. (2004). *Ground Penetrating Radar*. 2nd ed. IET.
 9. Soydan, O., et al. (2024). *On the utilization of the adjoint method in microwave tomography*.
 10. gprMax — Open-source GPR simulation software. <https://github.com/gprMax/gprMax>
-

Appendix A: Glossary of Terms

This glossary provides plain-language definitions of the specialised terms used throughout this document, organised alphabetically. Terms marked with *(physics)*, *(computing)*, or *(engineering)* indicate the primary domain.

Term	Definition
A-scan <i>(engineering)</i>	A single time-domain trace recorded at one transmitter–receiver position. It is a one-dimensional signal showing amplitude versus time. Multiple A-scans assembled side by side form a B-scan.
Adjoint method <i>(physics)</i>	A mathematical technique for computing how sensitive an output quantity (e.g., data misfit) is to every model parameter simultaneously, using only two simulations instead of one per parameter. Originates from optimal control theory.
Amplitude <i>(physics)</i>	The strength or magnitude of a wave signal at a given point in time or space.
B-scan <i>(engineering)</i>	A two-dimensional image (radargram) formed by stacking many A-scans recorded at successive scan positions. The horizontal axis is position, the vertical axis is time (or depth), and colour represents signal amplitude. Characteristic patterns such as hyperbolas reveal subsurface objects.
Bistatic <i>(engineering)</i>	A radar configuration where the transmitter and receiver are at separate (but usually nearby) locations, as opposed to monostatic (co-located).

Term	Definition
Boundary condition <i>(physics)</i>	A rule specifying how the electromagnetic fields behave at the edges of the computational domain. Without proper treatment, waves reflect artificially from the domain boundaries.
CFL condition <i>(computing)</i>	The Courant–Friedrichs–Lewy stability condition. A mathematical constraint on the time step size that ensures the numerical simulation remains stable (does not "blow up"). Named after three mathematicians who derived it in 1928.
Conductivity (σ) <i>(physics)</i>	A material property that quantifies how easily electric current flows through it, measured in siemens per metre (S/m). Metals have very high conductivity; concrete has low conductivity.
Convergence <i>(computing)</i>	The process by which an iterative algorithm progressively approaches a solution. A "converged" result is one where further iterations produce negligible improvement.
Courant number <i>(computing)</i>	A dimensionless number that relates the time step to the grid spacing and wave speed. Must be less than 1 for FDTD stability. Typical values are 0.9–0.95.
CPML <i>(computing)</i>	Convolutional Perfectly Matched Layer. An advanced absorbing boundary technique that uses auxiliary variables updated recursively to absorb outgoing waves at domain edges with minimal artificial reflection.
Cross-correlation <i>(physics)</i>	A mathematical operation that measures the similarity between two signals as a function of a time or space lag. In FWI, the cross-correlation of forward and adjoint wavefields produces the gradient.
CUDA <i>(computing)</i>	Compute Unified Device Architecture. NVIDIA's parallel computing platform that allows general-purpose programs to run on Graphics Processing Units (GPUs).
CuPy <i>(computing)</i>	An open-source Python library that provides a NumPy-compatible interface for GPU-accelerated array operations using CUDA.
Derivative-free optimisation <i>(computing)</i>	Optimisation methods (such as Nelder–Mead) that do not require computing the gradient (derivative) of the objective function. Useful when the objective is noisy or non-smooth.
Dielectric <i>(physics)</i>	An electrically insulating material (such as concrete or plastic) that can be polarised by an electric field. Dielectrics slow down electromagnetic waves proportionally to their permittivity.
Dispersion (numerical) <i>(computing)</i>	An artefact of the discrete grid where waves at different frequencies travel at slightly different speeds, distorting the pulse shape. Controlled by using enough grid points per wavelength.
Domain <i>(computing)</i>	The spatial region being simulated. In this work, a 500 mm × 300 mm rectangle representing a cross-section of a concrete slab.
FDTD <i>(computing)</i>	Finite-Difference Time-Domain. A numerical method that solves Maxwell's equations by replacing continuous derivatives with discrete differences on a grid, advancing the solution forward in time step by step.
Finite differences <i>(computing)</i>	A way of approximating a derivative (rate of change) by looking at the difference between function values at nearby points. For example, $df/dx \approx (f(x+h) - f(x-h))/(2h)$.
Forward simulation <i>(physics)</i>	Computing the predicted data (e.g., radar traces) from a known model (e.g., known material distribution). The "forward" direction goes from model to data.

Term	Definition
FWI (<i>physics</i>)	Full-Waveform Inversion. An advanced technique that uses the complete recorded waveform (not just travel times or amplitudes) to reconstruct the subsurface model.
GPU (<i>computing</i>)	Graphics Processing Unit. Originally designed for rendering images, modern GPUs contain thousands of small cores that can perform many calculations simultaneously, making them well-suited for scientific computing.
GPR (<i>engineering</i>)	Ground-Penetrating Radar. A geophysical method that uses radar pulses to image the subsurface. Applications include locating buried utilities, inspecting concrete structures, and archaeological surveys.
Grid cell (<i>computing</i>)	The smallest spatial unit of the computational mesh. In this work, each cell is a 2 mm × 2 mm square. All material properties and field values are defined per cell.
Hessian (<i>computing</i>)	The matrix of second derivatives of a function. In optimisation, approximating the Hessian (as L-BFGS-B does) allows faster convergence by accounting for the curvature of the objective landscape.
Hyperbola (<i>engineering</i>)	The characteristic bow-tie or inverted-V shape that appears in a B-scan when a point-like object (such as a rebar cross-section) is scanned. The shape arises because the object is at different ranges from different scan positions.
Ill-posedness (<i>physics</i>)	A property of inverse problems where the solution is non-unique or highly sensitive to noise. Regularisation is used to stabilise ill-posed problems.
Inverse problem (<i>physics</i>)	Determining the cause (model) from the observed effect (data). The opposite of the forward problem. In GPR: recovering the subsurface structure from recorded radar signals.
L-BFGS-B (<i>computing</i>)	Limited-memory Broyden-Fletcher-Goldfarb-Shanno with Bounds. A quasi-Newton optimisation algorithm that efficiently approximates the Hessian using gradient history and supports parameter bounds.
Lagrangian (<i>physics</i>)	A mathematical function that combines the objective to be minimised with constraints (such as the wave equation) using Lagrange multipliers. Used to derive the adjoint equations.
Leapfrog scheme (<i>computing</i>)	A time-integration method where two quantities (here E and H) are updated alternately, each offset by half a time step. This makes the scheme second-order accurate and energy-conserving.
Misfit (<i>physics</i>)	A scalar measure of how poorly the synthetic data matches the observed data. The inversion seeks to minimise the misfit. Commonly defined as the sum of squared differences.
Maxwell's equations (<i>physics</i>)	The four fundamental equations governing all electromagnetic phenomena, formulated by James Clerk Maxwell in 1865. They describe how electric and magnetic fields are generated and interact.
NDE / NDT (<i>engineering</i>)	Non-Destructive Evaluation / Non-Destructive Testing. Inspection techniques that assess the condition of a structure without causing damage. GPR is one such technique.
Nelder-Mead (<i>computing</i>)	A derivative-free optimisation algorithm (also called the "simplex method") that explores the parameter space using a geometric shape (simplex) that expands, contracts, and reflects to find the minimum.
NRMS (<i>computing</i>)	Normalised Root Mean Square error. A dimensionless error metric computed as the RMS of the difference between two quantities divided by

Term	Definition
	the RMS of the reference. Values closer to 0 indicate better agreement.
PEC (<i>physics</i>)	Perfect Electric Conductor. An idealised material with infinite conductivity that perfectly reflects all incident electromagnetic waves. Steel rebars behave approximately as PECs at GPR frequencies.
Permittivity (ϵ_r) (<i>physics</i>)	A material property that describes how much the material slows down electromagnetic waves relative to vacuum. Air has $\epsilon_r = 1$; dry concrete typically has $\epsilon_r \approx 4\text{--}10$. Also called the "dielectric constant".
PML (<i>computing</i>)	Perfectly Matched Layer. An artificial absorbing region placed around the computational domain to simulate an infinitely large space. Waves entering the PML are absorbed without reflection.
Radargram (<i>engineering</i>)	Another name for a B-scan image. A two-dimensional plot of GPR data showing reflections from subsurface features.
Rebar (<i>engineering</i>)	Short for "reinforcing bar". A steel bar embedded in concrete to provide tensile strength. Detecting and locating rebars is a common application of GPR in civil engineering.
Regularisation (<i>computing</i>)	A technique that adds extra information or constraints to an ill-posed problem to prevent overfitting and stabilise the solution. Total Variation (TV) regularisation promotes models with sharp boundaries.
Residual (<i>physics</i>)	The difference between observed data and the data predicted by the current model: $\mathbf{r} = \mathbf{d}_{\text{syn}} - \mathbf{d}_{\text{obs}}$. The inversion tries to make the residual as small as possible.
Ricker wavelet (<i>physics</i>)	A symmetric pulse shaped like a Mexican hat (the negative second derivative of a Gaussian). Commonly used as the source waveform in GPR and seismic simulations because it has zero DC component and well-defined bandwidth.
Simplex (<i>computing</i>)	In the context of Nelder–Mead optimisation, a geometric shape with $N + 1$ vertices in N -dimensional parameter space (e.g., a triangle in 2D, a tetrahedron in 3D). The algorithm manipulates this shape to find the minimum.
SIMT (<i>computing</i>)	Single Instruction, Multiple Threads. The execution model used by modern GPUs, where many threads execute the same instruction on different data elements simultaneously.
Soft source (<i>physics</i>)	A source injection technique where the source waveform is <i>added</i> to the existing field value ($E_z \leftarrow E_z + w(t)$), rather than <i>replacing</i> it. This avoids creating an artificial reflector at the source location.
Speedup (<i>computing</i>)	The ratio of execution time on one platform (e.g., CPU) to execution time on another (e.g., GPU). A speedup of $5\times$ means the GPU version runs 5 times faster.
Stencil (<i>computing</i>)	The pattern of neighbouring grid cells needed to update a given cell. The FDTD stencil reads from immediate neighbours only, which makes it highly parallelisable.
TMz (<i>physics</i>)	Transverse Magnetic with respect to the z -axis. A 2D electromagnetic mode where the electric field has only a z -component (E_z) and the magnetic field has x - and y -components (H_x, H_y).

Term	Definition
Total Variation (TV) <i>(computing)</i>	A regularisation technique that penalises the total amount of variation (gradient magnitude) in the model. It promotes piecewise-constant solutions with sharp boundaries — physically appropriate for concrete/rebar geometries.
Trace <i>(engineering)</i>	A single time-series recording of the received signal at one scan position. Synonym for A-scan.
Wavefield <i>(physics)</i>	The spatial distribution of the electromagnetic field (e.g., E_z values at every grid cell) at a particular instant in time.
Yee grid <i>(computing)</i>	The spatial discretisation scheme introduced by Kane Yee in 1966, where electric and magnetic field components are placed at staggered positions on the grid, offset by half a cell from each other.